

COMPLETE HTML5 FULL COURSE DOCUMENT

A detailed course from absolute beginner to modern HTML5 development

Includes: syntax, document structure, metadata, global attributes, text, links, URLs, images, responsive images, media, lists, tables, forms, input types, validation, semantics, layout, iframes, entities, paths, accessibility, responsive HTML, security, SEO, CSS/JS integration, browser APIs, obsolete HTML, projects, cheat sheet and practice questions.

Study method: read → type the example → open it in a browser → inspect the result → modify it.

HTML = STRUCTURE + MEANING

CSS = PRESENTATION • JavaScript = BEHAVIOR

TABLE OF CONTENTS

1. Fundamentals
2. Syntax
3. Document Structure
4. Head & Metadata
5. Global Attributes
6. Text
7. Links & URLs
8. Images
9. Audio & Video
10. Lists
11. Tables
12. Forms
13. Input Types
14. Validation
15. Semantic HTML
16. Layout / Div / Span
17. Iframes
18. Entities & Paths
19. Accessibility
20. Responsive HTML
21. Security
22. SEO
23. CSS & JavaScript Integration
24. Browser APIs
25. Obsolete HTML
26. Mistakes
27. Full Website
28. Projects
29. Cheat Sheet
30. Q&A;
31. Checklist

1. HTML Fundamentals

HTML = HyperText Markup Language. It is the standard markup language for structuring web documents. It is not a general-purpose programming language.

HyperText means documents/resources can be connected by links. **Markup** means content is annotated with elements describing structure and meaning.

Technology	Responsibility	Examples
HTML	Structure/meaning	headings, forms, images
CSS	Presentation	color, layout, spacing
JavaScript	Behavior/logic	events, DOM, API calls

The browser parses HTML into a document tree called the DOM. CSS can style that tree and JavaScript can interact with it.

```
<h1>Student Portfolio</h1>
<p>Information Technology student.</p>
<a href="/projects">View projects</a>
```

2. Syntax and Terminology

Element: the complete structure. **Tag:** markup notation. **Attribute:** additional information. **Content:** text/nested elements.

Nesting: elements inside elements. **Void element:** cannot contain children.

```
<p class="intro">Hello world.</p>
<a href="https://example.com">Visit</a>

```

Common void elements: img, br, hr, input, meta, link. Normal elements should be nested correctly and attribute values should be quoted.

Comments use `<!-- comment -->`. Never place secrets in HTML comments because users can inspect source.

3. Complete Document Structure

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>My Website</title>
</head>
<body>
  <header><h1>My Website</h1></header>
  <main><p>Hello!</p></main>
  <footer><p>Copyright</p></footer>
</body>
</html>
```

- 1 DOCTYPE requests standards mode.
- 2 html is the root element.
- 3 head contains metadata/resources.
- 4 body contains document content.
- 5 lang declares language and helps accessibility/tools.

Indentation is for readability; nesting creates the structure.

4. Head and Metadata

Element	Purpose	Example
title	Document/tab title	<title>Home</title>
meta	Metadata	<meta charset="utf-8">
link	External resource	<link rel="stylesheet" href="style.css">
style	Embedded CSS	<style>...</style>
script	JavaScript	<script src="app.js" defer>
base	Base URL	<base href="https://example.com/">

charset: UTF-8 is the normal modern encoding. **viewport:** helps mobile browsers use the intended viewport. **description:** useful metadata for search presentation. **canonical:** can identify a preferred URL.

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Learn HTML step by step.">
  <title>HTML Course</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
```

defer lets an external script download while parsing and execute after parsing.

5. Global Attributes

Attribute	Purpose	Example
id	Unique identifier	id="contact"
class	Reusable grouping	class="card"
title	Advisory information	title="Details"
lang	Language	lang="ur"
dir	Text direction	dir="rtl"
hidden	Currently not relevant	hidden
tabindex	Keyboard focus	tabindex="0"
style	Inline CSS	style="color:red"
data-*	Custom data	data-id="42"
contenteditable	Editable content	contenteditable="true"
spellcheck	Spellcheck preference	spellcheck="true"

IDs should be unique within a document; classes can be reused. data-* attributes are for application-specific data.

6. Text, Headings and Formatting

h1 through h6 represent heading hierarchy. Use CSS for visual sizing rather than selecting headings only for their default font size.

```
<h1>Web Development</h1>
<h2>HTML</h2>
<h3>Forms</h3>
<p>A paragraph.</p>
<p>Line one<br>Line two</p>
<hr>
```

Element	Meaning
strong	Strong importance
em	Stress emphasis
b	Attention without strong-importance semantics
i	Alternate voice/mood/technical context
mark	Relevant highlight
small	Side comments/small print
del / ins	Deleted / inserted
sub / sup	Sub/superscript
code	Inline code
kbd	Keyboard input
samp	Sample output
var	Variable
abbr	Abbreviation
cite	Title of a work
blockquote / q	Long / short quotation
pre	Preformatted text

Use `br` for a real line break, not for layout spacing; use CSS for spacing.

7. Links and URLs

The anchor element creates hyperlinks. `href` identifies the destination.

```
<a href="https://example.com">Absolute URL</a>
<a href="/about.html">Root-relative URL</a>
<a href="images/photo.jpg">Relative URL</a>
<a href="#contact">Fragment</a>
<a href="mailto:hello@example.com">Email</a>
<a href="tel:+923001234567">Phone</a>
```

Part	Example	Meaning
Scheme	https	Access scheme
Host	example.com	Hostname
Port	443	Optional transport port
Path	/about	Resource/path
Query	?page=2	Parameters
Fragment	#contact	Client-side target

A fragment is handled by the browser and is not normally sent to the server. `target="_blank"` commonly opens another browsing context; `rel="noopener"` is a common defensive choice.

8. Images and Responsive Images

```

```

src identifies the image. alt provides the appropriate text alternative. Meaningful images need useful alt; decorative images generally use alt="".

```

```

srcset supplies candidates and sizes describes expected display width. picture is useful for art direction.

```
<picture>
  <source media="(max-width: 600px)" srcset="mobile.jpg">
  
</picture>
```

Avoid redundant alt text when nearby text already communicates the same information.

9. Audio and Video

```
<audio controls>
  <source src="lesson.mp3" type="audio/mpeg">
</audio>

<video controls width="640" poster="poster.jpg">
  <source src="lesson.mp4" type="video/mp4">
</video>
```

Important attributes include controls, autoplay, muted, loop, poster and preload. Browsers may restrict audible autoplay.

```
<video controls>
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions.vtt"
        kind="captions"
        srclang="en"
        label="English">
</video>
```

Captions and transcripts improve accessibility.

10. Lists

Element	Purpose
ul	Unordered list
ol	Ordered list
li	List item
dl	Description list
dt	Term
dd	Description/details

```
<ul><li>HTML</li><li>CSS</li></ul>
<ol><li>Create file</li><li>Write HTML</li></ol>
<dl><dt>HTML</dt><dd>Structures content.</dd></dl>
```

Ordered lists support attributes such as start and reversed. CSS controls marker appearance.

11. Tables

Use tables for tabular data, not page layout.

```
<table>
  <caption>Student Results</caption>
  <thead>
    <tr><th scope="col">Name</th><th scope="col">CGPA</th></tr>
```

```
</thead>
<tbody>
  <tr><td>Ali</td><td>3.7</td></tr>
  <tr><td>Sara</td><td>3.9</td></tr>
</tbody>
</table>
```

Element	Purpose
table	Container
caption	Title/description
thead/tbody/tfoot	Row groups
tr	Row
th	Header
td	Data
colgroup/col	Column definitions

scope="col" or scope="row" helps communicate header relationships to assistive technologies.

12. Forms

```
<form action="/register" method="post">
  <label for="name">Name</label>
  <input id="name" name="name" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <button type="submit">Create account</button>
</form>
```

Concept	Meaning
action	Submission URL
method	Usually GET or POST
name	Submitted field name
value	Field value
label	Accessible name
button	Action/submission control

GET commonly puts data in the URL query; POST commonly sends it in the request body. HTTPS is required for sensitive data.

13. Input Types

Type	Typical use
text	General text
password	Password
email	Email
url	URL
tel	Telephone
search	Search
number	Number
range	Slider
date	Date
month	Month
week	Week
time	Time
datetime-local	Local date/time
color	Color
checkbox	Independent choice
radio	One choice in group
file	File selection
hidden	Hidden submitted value; not security
submit	Submit
reset	Reset
button	Generic button
image	Image submit

```

<input type="email" name="email" required>
<input type="number" name="age" min="18" max="100">
<input type="date" name="dob">
<input type="checkbox" name="terms" required>
<input type="radio" name="plan" value="basic">
<input type="radio" name="plan" value="pro">
<input type="file" name="resume">

```

Radio buttons form a group when they share a name. Checkboxes represent independent selections.

14. Validation

HTML provides built-in constraint validation, but browser validation is not a security boundary. The server must validate and authorize data.

Attribute	Purpose
required	Must have a value
minlength / maxlength	Text length
min / max	Bounds
step	Allowed increments

Attribute	Purpose
pattern	Pattern constraint
multiple	Multiple values where supported
accept	File-type hint
autocomplete	Autofill hint

```
<input name="username"
      minlength="3" maxlength="20" required>
<input name="age" type="number"
      min="18" max="100" required>
```

Never trust hidden fields for prices, roles, permissions or other sensitive values.

15. Semantic HTML

Element	Meaning
header	Introductory content
nav	Navigation
main	Dominant main content
section	Thematic section
article	Self-contained composition
aside	Related content
footer	Footer
figure	Self-contained media/content
figcaption	Figure caption
details	Disclosure widget
summary	Visible details label
time	Date/time
dialog	Dialog

Semantic HTML improves meaning, accessibility and maintainability. Choose elements because their meaning fits, not because their default appearance looks good.

16. Layout, Div and Span

```
<body>
  <header>
    <h1>My Site</h1>
    <nav aria-label="Main navigation">
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>
  <main>
    <article>
      <h2>HTML Course</h2>
      <section><h3>Intro</h3><p>HTML structures content.</p></section>
    </article>
    <aside><h2>Related</h2></aside>
  </main>
  <footer><p>Copyright</p></footer>
</body>
```

div is a generic block container. span is a generic inline container. Use them when no more meaningful semantic element exists.

```
<div class="card">
  <h2>Product</h2>
  <p>Price: <span class="price">$50</span></p>
</div>
```

17. Iframes and Embedding

```
<iframe
  src="https://example.com"
  title="Example content"
  width="800" height="500"
  loading="lazy">
</iframe>
```

iframe creates an embedded browsing context. Give it a useful title. Third-party content can introduce security, privacy and performance concerns.

Use specific elements such as `img`, `audio` or `video` when they accurately describe the embedded content.

18. Entities and File Paths

Character	Reference
<	<
>	>
&	&
"	"
©	©
non-breaking space	

To display literal HTML markup as text, use character references such as `<` and `>`.

Path	Example	Meaning
Same directory	photo.jpg	Current directory
Child	images/photo.jpg	Inside images
Parent	../photo.jpg	One level up
Root-relative	/images/photo.jpg	From site root
Absolute	https://cdn.example.com/a.jpg	Complete URL

Browsers request URLs. A Windows filesystem path on your computer is not automatically a web URL.

19. Accessibility

Accessibility means making content usable by people with different abilities, including keyboard and assistive-technology users.

- Use semantic HTML before ARIA.
- Set the correct lang.
- Give informative images useful alt text; decorative images generally use `alt=""`.
- Label form controls.
- Use buttons for actions and links for navigation.
- Maintain logical headings.
- Keep interactive elements keyboard-operable.

- Preserve visible focus.
- Provide captions/transcripts where appropriate.
- Mark table headers correctly.

Native HTML is powerful. A real button already has expected semantics and keyboard behavior. ARIA should add semantics when native HTML cannot express the needed behavior.

20. Responsive HTML

```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
```

Responsive design is mainly CSS, but HTML should provide flexible semantic structure and responsive assets.

- Use semantic structure.
- Use srcset/sizes for image candidates.
- Avoid layout tables.
- Use scalable media.
- Make controls usable on touch and keyboard.

The viewport tag does not itself make a website responsive; CSS media queries and flexible layouts do the visual work.

21. Security Fundamentals

- Never put secrets in HTML; client-delivered content can be inspected.
- Hidden inputs are not secure and can be modified.
- Use HTTPS for protected transport.
- Validate and authorize on the server.
- Escape untrusted output when generating HTML.
- Use appropriate cookie security attributes.
- Review third-party scripts and embedded content.

```
<!-- Not a secure price store -->
<input type="hidden" name="price" value="10">
```

A user can modify that hidden value. The server must determine trusted prices/permissions instead.

22. SEO

- Use a meaningful title.
- Use clear headings and semantic structure.
- Use descriptive link text.
- Use appropriate alt text.
- Write useful content.
- Use a meta description when appropriate.
- Use canonical URLs when needed.
- Use accurate structured data where appropriate.

```
<head>
  <title>HTML Forms – Complete Guide</title>
```

```
<meta name="description"
      content="Learn HTML forms and validation.">
<link rel="canonical"
      href="https://example.com/html/forms">
</head>
```

Good HTML helps machines understand content but does not guarantee rankings.

23. HTML + CSS + JavaScript

```
<!doctype html>
<html lang="en">
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <button id="hello">Say hello</button>
  <script src="app.js" defer></script>
</body>
</html>
```

Learning model: HTML = structure/meaning; CSS = presentation; JavaScript = behavior. Build meaningful HTML before styling and scripting.

Progressive enhancement: useful HTML first, CSS second, JavaScript enhancement third.

24. Browser/Web APIs Overview

Feature/API	Role
DOM	JavaScript representation of the document
Fetch	Network requests
Web Storage	localStorage/sessionStorage
Geolocation	Location with permission
Canvas	Scriptable drawing surface
WebSocket	Persistent bidirectional communication
History API	Browser history
Clipboard API	Clipboard access
Web Workers	Background JS contexts
Service Workers	Network/cache/offline layer

These are commonly learned with HTML but are not all HTML tags. Canvas is an element; drawing is performed through JavaScript APIs.

25. Deprecated / Obsolete HTML

Avoid old presentation/layout elements in new projects.

Old approach	Modern approach
font / center	CSS
strike	s or del depending on meaning
tt	code or appropriate semantic text
frameset/frame	Modern layout; iframe only for real embedding

Old approach	Modern approach
marquee	Avoid; use accessible modern interaction if genuinely required

26. Common Mistakes

Mistake	Better approach
Missing doctype	Use <!doctype html>
Missing lang	Set html lang
Image without alt	Provide appropriate alt
Input without label	Use label + for/id
Div for everything	Use semantic elements
Table for layout	Use CSS layout
br for spacing	Use CSS
Duplicate IDs	Keep IDs unique
Trust client validation	Validate server-side
Secret in HTML	Keep secrets server-side
Huge images	Optimize/responsive images

27. Complete Website Example

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Student portfolio">
  <title>Student Portfolio</title>
</head>
<body>
<header>
  <h1>Student Portfolio</h1>
  <nav aria-label="Main navigation">
    <a href="#about">About</a>
    <a href="#projects">Projects</a>
    <a href="#contact">Contact</a>
  </nav>
</header>

<main>
  <section id="about">
    <h2>About</h2>
    <p>I am an IT student learning web development.</p>
  </section>

  <section id="projects">
    <h2>Projects</h2>
    <article>
      <h3>Portfolio Website</h3>
      <p>A semantic HTML project.</p>
    </article>
  </section>

  <section id="contact">
    <h2>Contact</h2>
    <form action="/contact" method="post">
      <label for="name">Name</label>
      <input id="name" name="name" required>
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required>
      <label for="message">Message</label>
      <textarea id="message" name="message" rows="5" required></textarea>
      <button type="submit">Send</button>
    </form>
  </section>
</main>

<footer><p>© 2026 Student Portfolio</p></footer>
</body>
</html>
```

Build this with HTML only first, then add CSS, then JavaScript where actual behavior is needed.

28. Mini Projects

Project	Practice
Personal profile	Headings, paragraphs, image, links, lists
Portfolio	Semantic layout, navigation, articles
Resume	Lists, links, contact
Registration form	Labels, inputs, validation
Product page	Images, sections, tables
Blog	article, figure, blockquote, time
Course site	Navigation, sections, tables, forms
Accessible contact	A11y, labels, landmarks
Media page	Audio, video, captions
Full portfolio	Combine all concepts

Recommended sequence: HTML-only → CSS → JavaScript. This keeps the responsibility of each technology clear.

29. HTML Cheat Sheet

Document

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page</title>
</head>
<body></body>
</html>
```

Structure

```
header nav main section article aside footer
```

Text

```
h1-h6 p strong em mark small code kbd
blockquote q pre br hr del ins sub sup
```

Media

```
a img picture source audio video track iframe
```

Lists

```
ul ol li dl dt dd
```

Tables

```
table caption thead tbody tfoot tr th td
```

Forms

```
form label input textarea select option
optgroup button fieldset legend datalist output
```

High-value attributes

```
id class lang dir title hidden tabindex data-*
href src alt width height name value type action method
target rel required placeholder disabled readonly checked
selected min max minlength maxlength pattern autocomplete accept
```

30. Practice Q&A;

1. **What is HTML?** HyperText Markup Language for structuring web content.
2. **Is HTML a programming language?** No; it is a markup language.
3. **What is DOM?** A programmatic representation of the document tree.
4. **Why doctype?** To request standards mode.
5. **Why lang?** It helps browsers, assistive technology and language tools.
6. **Why alt?** It provides an appropriate image text alternative.
7. **Why label?** It gives a form control a clear accessible name.
8. **GET vs POST?** GET commonly puts form data in the URL; POST commonly sends it in the body.
9. **Semantic HTML?** Choosing elements according to meaning.
10. **Is hidden input secure?** No; the client can change it.
11. **Is client validation enough?** No; validate on the server.
12. **What is srcset?** Multiple image candidates for browser selection.
13. **What are tables for?** Tabular data, not page layout.
14. **Link vs button?** Link navigates; button performs an action.
15. **What does required do?** Adds a required-value constraint.
16. **What is iframe?** An embedded browsing context.
17. **Progressive enhancement?** Useful HTML first, then CSS, then JS enhancement.
18. **Why avoid deprecated HTML?** Modern semantic HTML/CSS is preferable.
19. **Accessibility?** Making content usable by people with different abilities.
20. **Three basic layers?** HTML structure, CSS presentation, JavaScript behavior.

31. Final Checklist

- Explain element, tag, attribute, content, nesting and void elements.
- Write a complete HTML document from memory.
- Use metadata correctly.
- Use global attributes correctly.
- Create a meaningful heading hierarchy.
- Use semantic text elements.
- Create absolute, relative and fragment links.
- Use images with correct alt text.
- Use srcset/picture.
- Embed audio/video and captions.
- Build lists and accessible tables.
- Create forms with labels and correct input types.
- Use validation constraints correctly.
- Use fieldset/legend.
- Build semantic layouts.

- Know when div/span are appropriate.
- Understand iframe/third-party content.
- Use entities and relative paths.
- Apply accessibility basics.
- Understand responsive HTML vs CSS.
- Keep secrets out of client HTML.
- Validate/authorize on the server.
- Use SEO-friendly structure.
- Connect HTML to CSS/JS.
- Know major browser APIs.
- Recognize obsolete HTML.
- Build at least three complete projects.

Final mental model

HTML gives structure and meaning → CSS gives presentation → JavaScript gives behavior. Strong HTML is semantic, accessible, maintainable and understandable before styling is added.

Next learning path: CSS → selectors/cascade → box model → display → Flexbox → Grid → responsive design → typography → animations → projects; then JavaScript → DOM → events → forms → APIs → applications.